

Laporan Tugas Besar Strategi Algoritma

Anggota Kelompok 8 RC:
Galih Sigit Satrio (124140001)
Christoper Leon Saputra (124140097)
Kenzie Sahasika Tariana (124140103)



Program Studi Teknik Informatika
Institut Teknologi Sumatera

Daftar Isi	
Bab 1. Deskripsi Tugas	3
Bab 2. Landasan Teori	4
A. Dasar Teori	4
B. Cara Kerja Bot	4
Bab 3. Aplikasi Strategi Greedy	5
A. Mapping Persoalan Robocode Tank Royale	5
B. Eksplorasi Alternatif	5
1. Strategi 1 LahBebas: State Machine Adaptif	5
2. Strategi 2 BotBebas: Gerakan Segitiga di Tembok	6
3. Strategi 3 BotGladiator: Modified Gerakan Segitiga di Tembok	7
4. Strategi 4: Imo: Wall Patrol dengan Greedy Defensif	7
Bab 4. Implementasi dan Pengujian	8
Bab 5. Lampiran dan Saran	9
Lampiran	10

Bab 1. Deskripsi Tugas

- Buatlah 4 bot (1 utama dan 3 alternatif, tetap dikumpulkan 1 bot yang terbaik saja) dalam bahasa C# (.net) yang mengimplementasikan *algoritma Greedy* pada *bot* permainan Robocode Tank Royale dengan tujuan memenangkan permainan.
- Tugas dikerjakan berkelompok dengan anggota minimal 2 orang dan maksimal 3 orang.
- Strategi *greedy* yang diimplementasikan setiap kelompok harus dikaitkan dengan fungsi objektif dari permainan ini, yaitu memperoleh skor setinggi mungkin pada akhir pertempuran. Hal ini dapat dilakukan dengan mengoptimalkan komponen skor yang telah dijelaskan diatas.
- Strategi *greedy* yang diimplementasikan harus berbeda untuk setiap bot yang diimplementasikan dan setiap strategi *greedy* harus menggunakan heuristic yang berbeda.
- Bot yang dibuat TIDAK BOLEH sama dengan SAMPEL yang diberikan sebagai CONTOH. Baik dari starter pack maupun dari repository engine asli.
- Buatlah strategi *greedy* terbaik, karena setiap bot utama dari masing-masing kelompok akan diadu dalam kompetisi Tubes.
- Strategi *greedy* yang kelompok anda buat harus dijelaskan dan ditulis secara eksplisit pada laporan, karena akan diperiksa saat demo apakah strategi yang dituliskan sesuai dengan yang diimplementasikan.
- Setiap kelompok dapat menggunakan kreativitas yang bermacam macam dalam menyusun strategi *greedy* untuk memenangkan permainan. Implementasi pemain harus dapat dijalankan pada *game engine* yang telah disebutkan diatas serta dapat dikompetisikan dengan bot dari kelompok lain.
- Program harus mengandung komentar yang jelas, dan untuk setiap strategi *greedy* yang disebutkan, harus dilengkapi dengan kode sumber yang dibuat. Artinya semua strategi harus diimplementasikan
- Mahasiswa disarankan membaca dokumentasi dari *game engine*. Perlu diperhatikan bahwa game engine yang digunakan untuk tubes ini SUDAH DIMODIFIKASI. Untuk Perubahan dapat dilihat pada bagian starter pack.

Bab 2. Landasan Teori

A. Dasar Teori

Algoritma Greedy adalah pendekatan pemecahan masalah yang mengikuti heuristik membuat pilihan terbaik secara lokal di setiap tahap dengan harapan menemukan optimum global.

B. Cara Kerja Bot

Bot utama menggunakan strategi umum sebagai berikut:

- Bot menggunakan Finite State Machine (FSM) dengan 5 state:
 - HUNT → mencari musuh
 - FIGHT → bertarung (jarak < 600)
 - FLEE → kabur (energi < 15)
 - DODGE → menghindari peluru
 - UNSTUCK → keluar dari posisi macet
- Setiap turn, bot mengevaluasi state secara greedy — langsung memilih state pertama yang kondisinya terpenuhi
- Menggunakan greedy scoring — memilih musuh dengan score terkecil (bukan terbaik secara global)
- Memilih firepower berdasarkan kondisi sekarang (jarak & energi musuh), dan langsung menembak saat kondisi terpenuhi.

Bab 3. Aplikasi Strategi Greedy

Mapping Persoalan Robocode Tank Royale

1. Himpunan Kandidat (C)
 $C = \{$
Gerakan : maju (n px), mundur (n px), belok kanan/kiri (θ°),
Gun : putar gun kanan/kiri (θ°), tembak (power 1.0 / 2.0 / 3.0),
Radar : putar radar kanan/kiri (sweep / lock),
State : HUNT, FIGHT, FLEE, DODGE, UNSTUCK (khusus LahBebas),
Patrol : maju ke tembok, sapukan gun, pindah tembok (khusus wall bots),
Kombinasi: semua sub-aksi dapat dieksekusi paralel dalam 1 turn
 $\}$
2. Himpunan Solusi (S)
 $S = \{ \text{aksi_turn_1, aksi_turn_2, ..., aksi_turn_n} \}$
Berhenti saat: menang / kalah
3. Fungsi Solusi
Per-turn: Selesai saat semua sistem diset dan Go () dipanggil
Pertandingan: Selesai saat semua musuh eliminasi atau bot sendiri mati
4. Fungsi Objektif
Utama : Maksimalkan jumlah musuh yang dieliminasi (kill count)
Per-turn : Minimasi (error sudut gun + jarak ke musuh + area tidak tercakup radar)

Eksplorasi Alternatif

1. Strategi 1 LahBebas: State Machine Adaptif

Bot menggunakan Finite State Machine (FSM) dengan 5 state (HUNT, FIGHT, FLEE, DODGE, UNSTUCK). Setiap turn dievaluasi secara greedy — state prioritas tertinggi yang kondisinya terpenuhi langsung dipilih dan dieksekusi.

Heuristik:

Pilih state dengan prioritas tertinggi yang kondisinya terpenuhi saat ini, lalu eksekusi aksi terbaik untuk state tersebut secara independen untuk radar, gun, dan movement.

Fungsi Seleksi:

State : priority combatOverride → UNSTUCK → DODGE → FLEE → FIGHT → HUNT

Target : argmin(dist + staleness × 50) dari semua musuh terlihat dalam 15 turn

Firepower: threshold jarak → <150: 3.0 | <300: 2.0 | <500: 1.5 | else: 1.0

Fire timing: tembak jika |gunTurn| < 5°, atau force-fire jika 30 turn tidak tembak

Gerakan : Tegak lurus terhadap musuh di state FIGHT.

Fungsi Kelayakan:

Tembak : GunHeat == 0 AND ada target AND threshold sudut terpenuhi

FIGHT : distance < 600 AND Energy >= 15 AND tidak

unstucking/dodging Prediksi : staleness musuh < 8 turn (data masih fresh)

Menghindari tembok: wallCooldown == 0 AND (bot dalam margin 100px dari dinding)

2. Strategi 2 BotBebas: Gerakan Segitiga di Tembok

Bot bergerak ke tembok terdekat, lalu berpatroli bolak-balik sepanjang 30% lebar/tinggi arena di tepi tembok tersebut. Selama bergerak, gun gerakkan $\pm 90^\circ$ (total 180°) menghadap ke dalam arena. Jika menabrak musuh, bot langsung pindah ke tembok seberang. Namun karena posisi spawn bot yang random bot tidak dapat selalu berada di area tembok, sehingga bot bergerak segitiga secara random di tengah arena.

Heuristik:

Selalu mencoba mendekati tembok aktif dan gerakkan gun 180° ke dalam arena, karena ide awal tembok membatasi arah datang musuh sehingga hanya 180° yang perlu dipindai.

Fungsi Seleksi:

Tembok tujuan : tembok Utara (default awal), lalu seberang (currentWall + 2) saat nabrak musuh

Patrol arah : tembok berikutnya searah jarum jam (nextWall = currentWall + 1)

Jarak patrol : 30% lebar arena (horizontal) atau 30% tinggi arena (vertikal)

Firepower : dist > 200 → Fire(1) | dist > 50 → Fire(2) | else → Fire(3)

Gun sweep : 3° /tick, balik arah di $\pm 90^\circ$

Fungsi: Kelayakan:

Tembak : OnScannedBot terpanggil (radar langsung tembak)

Pindah tembok : hitEnemyFlag == true

Lanjut patrol : hitEnemyFlag == false AND DistanceRemaining > 0.5

Anti-stuck : stuck > 5 tick berturut-turut → paksa keluar loop

3. Strategi 3 BotGladiator: Modified Gerakan Segitiga di Tembok

Merupakan varian agresif dan tidak terprediksi dari BotBebas. Bot bergerak ke tembok dengan pola zigzag awal. Setelah di tembok, patrol mencakup 90% dimensi arena, dan di dalam pergerakan patrol terdapat zigzag tambahan sebelum gun sweep dimulai. Namun karena posisi spawn bot yang random bot tidak dapat selalu berada di area tembok, dan melakukan pergerakan di tengah map.

Heuristik:

Tutup area arena seluas mungkin dengan gerakan zigzag yang tidak terprediksi menyebabkan musuh sulit mengantisipasi posisi bot berikutnya, sehingga probabilitas tertembak menurun.

Fungsi Seleksi:

Tembok tujuan : seberang ($\text{currentWall} + 2$) digunakan untuk langsung lompat jauh

Patrol arah : $\text{nextWall} = (\text{currentWall} + 2) \% 4$

Jarak patrol : 90% dimensi arena (vs 30% BotBebas)

Zigzag awal : TurnRight(30) → Forward(250) → TurnLeft(90) → Forward(250)

Zigzag dalam : TurnRight(30) → Forward(500) → TurnLeft(60) → Forward(500)

Firepower : sama seperti BotBebas (berbasis jarak) Gun sweep : $3^\circ/\text{tick}$ dengan batas $\pm 270^\circ$ (lebih lebar dari $\pm 90^\circ$ BotBebas)

Fungsi Kelayakan:

Tembak : OnScannedBot terpanggil

Zigzag : dieksekusi tanpa kondisi — selalu dilakukan

Pindah tembok : $\text{hitEnemyFlag} == \text{true} \rightarrow (\text{currentWall} + 2) \% 4$

Anti-stuck : stuck > 2 tick → paksa keluar (lebih agresif dari BotBebas)

4. Strategi 4: Imo: Wall Patrol dengan Greedy Defensif

Secara struktural mirip BotBebas (patrol 30%), namun dengan tambahan greedy firepower yang mempertimbangkan energi sendiri, respons defensif saat kena tembakan (dodge 30° + forward 50px), dan koreksi saat menabrak dinding (belok 10°)

Heuristic:

Patroli tembok dengan greedy firepower yang mempertimbangkan kondisi diri sendiri (energi) selain kondisi musuh (jarak) — selamatkan diri dulu sebelum

menyerang. Namun karena posisi spawn bot yang random bot tidak dapat selalu berada di area tembok, dan melakukan pergerakan di tengah map.

Fungsi Seleksi:

Tembok tujuan : Utara (default), seberang saat menabrak musuh (currentWall + 2)

Patrol arah : nextWall = currentWall + 1 (searah jarum jam)

Jarak patrol : 30% dimensi arena

Firepower : Energy < 15 → Fire(1), cek energi dahulu jika mencukupi lakukan pengecekan berikut dist > 200 → Fire(1) dist > 50 → Fire(2) else → Fire(3)

Dodge : OnHitByBullet → TurnRight(30) + Forward(50)

Wall correction: OnHitWall → TurnRight(10)

Fungsi Kelayakan:

Tembak : OnScannedBot terpanggil

Dodge : OnHitByBullet terpanggil (reaktif, bukan proaktif)

Wall correct : OnHitWall terpanggil

Pindah tembok : hitEnemyFlag == true Anti-stuck : stuck > 5 tick → paksa keluar loop

Analisis per Strategi

LahBebas

Efisiensi: Sedang. Evaluasi state setiap turn $O(n)$ untuk scan musuh, namun semua operasi setelahnya konstan. Overhead paling tinggi dari keempat bot, namun tetap dalam batas komputasi yang sangat ringan untuk Robocode.

Efektivitas: Tinggi. Satu-satunya bot yang memiliki state survival (FLEE), dodge aktif (DODGE), dan anti-stuck komprehensif (UNSTUCK 3 fase). Mampu menghadapi berbagai skenario pertandingan secara dinamis.

Kelemahan: Kompleksitas kode tinggi meningkatkan risiko interaksi antar state yang tidak diinginkan.

BotBebas

Efisiensi: Sangat tinggi. Tidak ada kalkulasi kompleks, hanya pergerakan linear dan gun sweep konstan. Paling ringan secara komputasi.

Efektivitas: Sedang. Posisi di tepi dinding memang melindungi satu sisi, namun pola patrol 30% sangat terprediksi. Musuh yang bergerak aktif dapat dengan mudah menghindari gun sweep yang bergerak lambat ($3^\circ/\text{tick}$).

Kelemahan: Tidak ada dodge, tidak ada survival mode, pola sangat dapat diantisipasi musuh. Spawn posisi yang tidak bisa ditentukan membuat pergerakan menuju tembok tidak dapat terjadi secara konsisten.

BotGladiator

Efisiensi: Rendah-Sedang. Banyak gerakan tidak perlu (zigzag ganda setiap siklus patrol) yang menghabiskan waktu tanpa manfaat taktis yang jelas.

Efektivitas: Rendah. Terdapat **nilai-nilai yang tidak valid** dalam kode: `gunDir = -20.0` dan `gunDir = 180.0` menyebabkan gun sweep tidak bekerja sebagaimana mestinya. Patrol 90% arena menyebabkan bot sering berbalik jauh dari posisi ideal. Zigzag awal sebelum ke tembok juga tidak memberikan manfaat taktis yang terukur.

Kelemahan: Bug pada nilai `gunDir` membuat gun sweep tidak terkontrol. Merupakan strategi paling tidak stabil dari keempat bot. Spawn posisi yang tidak bisa ditentukan membuat pergerakan menuju tembok tidak dapat terjadi secara konsisten.

Imo

Efisiensi: Tinggi. Struktur kode paling bersih, logika identik dengan BotBebas namun dengan tambahan minimal yang tidak meningkatkan overhead secara signifikan.

Efektivitas: Sedang-Tinggi. Penambahan cek energi dalam pemilihan firepower adalah penambahan yang meningkatkan efisiensi energi. Respons terhadap `OnHitByBullet` dan `OnHitWall` memberikan sedikit kemampuan defensif. Namun masih terbatas oleh pola patrol yang terprediksi.

Kelemahan: Dodge hanya reaktif (setelah kena), bukan proaktif. Pola gerakan masih dapat diantisipasi musuh yang agresif.

Strategi yang Dipilih

Bot LahBebas merupakan bot yang dipilih untuk implementasi dan pengumpulan, alasan utamanya adalah bahwa LahBebas merupakan satu-satunya bot yang mengimplementasikan greedy di empat lapisan berbeda secara bersamaan. Selain itu LahBebas menggunakan scoring dua dimensi hal ini mempertimbangkan seberapa dekat sekaligus seberapa fresh data musuh. Terakhir LahBebas memiliki mekanisme anti-stuck yang lebih baik dibandingkan bot bot yang lain

Bab 4. Implementasi dan Pengujian

A. Implementasi Pseudo-Code

KELAS: EnemyInfo

Menyimpan data musuh hasil scan radar

STRUKTUR EnemyInfo:

Id, X, Y, PrevX, PrevY // posisi sekarang dan sebelumnya
Direction, Speed, Energy // arah, kecepatan, energi musuh
PrevEnergy, LastSeen // energi sebelumnya, turn terakhir terlihat

VelocityX = X - PrevX // estimasi kecepatan sumbu X
VelocityY = Y - PrevY // estimasi kecepatan sumbu Y
JustFired = (PrevEnergy - Energy) antara 0.09 dan 3.1
 // deteksi musuh baru saja menembak

ENUM BotState: HUNT, FIGHT, FLEE, DODGE, UNSTUCK

Lima state yang menentukan perilaku bot setiap turn

PROSEDUR UTAMA: Run()

Main loop — dijalankan terus selama bot hidup

Setiap iterasi = 1 turn pertandingan

PROSEDUR Run():

Atur warna bot

Atur independensi gun dan radar terhadap body

SELAMA bot masih hidup:

// -- TICK SEMUA SISTEM --

UpdateStuckDetection() // cek apakah bot diam terlalu lama

UpdateStateTimeout() // cek apakah bot macet di satu state

UpdateCombatOverride() // hitung countdown combat override

kurangi wallCooldown jika > 0

tambah turnsSinceLastFire

```

// -- GREEDY: EVALUASI KONTEKS --
nearestEnemy ← GetNearestEnemy() // pilih target terbaik saat ini
currentState ← EvaluateState()    // pilih state prioritas tertinggi

// -- SISTEM INDEPENDEN --
RadarLogic()          // selalu update radar tiap turn
GunTrackingLogic()     // selalu update arah gun tiap turn

// -- MOVEMENT SESUAI STATE --
JIKA state == UNSTUCK → UnstuckRoutine()
JIKA state == FIGHT  → FightMovement()
JIKA state == FLEE   → FleeMovement()
JIKA state == DODGE  → DodgeMovement()
DEFAULT              → HuntMovement()

// -- FIRE: INDEPENDEN DARI MOVEMENT --
FireLogic()

Go() // eksekusi semua perintah sekaligus (1 tick)

simpan posisi sekarang ke prevX, prevY

```

FUNGSI: EvaluateState()

GREEDY — Pilih state dengan prioritas tertinggi
yang kondisinya terpenuhi saat ini

FUNGSI EvaluateState() → BotState:

JIKA combatOverride aktif DAN ada nearestEnemy:

KEMBALIKAN FIGHT // prioritas 1: paksa tempur setelah unstuck

JIKA isUnstucking == true:

KEMBALIKAN UNSTUCK // prioritas 2: keluar dari macet

JIKA dodging == true DAN dodgeTurns > 0:

kurangi dodgeTurns

JIKA dodgeTurns habis → dodging = false

KEMBALIKAN DODGE // prioritas 3: hindari peluru aktif

JIKA Energy < 15:

KEMBALIKAN FLEE // prioritas 4: selamatkan diri

JIKA nearestEnemy != null DAN jarak < 600:
 KEMBALIKAN FIGHT // prioritas 5: musuh dalam jangkauan

KEMBALIKAN HUNT // default: cari musuh

FUNGSI: GetNearestEnemy()
 GREEDY — Pilih musuh dengan score minimum
 Score menggabungkan jarak dan kesegaran data

FUNGSI GetNearestEnemy() → EnemyInfo:

 best ← null
 bestScore ← MAX_VALUE

 UNTUK SETIAP musuh dalam enemies:
 JIKA musuh tidak terlihat > 15 turn → lewati

 dist ← jarak Euclidean ke musuh
 staleness ← TurnNumber - musuh.LastSeen

 score ← dist + (staleness × 50)
 // Musuh dekat + baru terlihat = score rendah = prioritas tinggi

 JIKA score < bestScore:
 bestScore ← score
 best ← musuh

 KEMBALIKAN best

PROSEDUR: RadarLogic()
 Radar selalu independen — lock musuh jika terlihat,
 sweep 45° jika tidak ada musuh

PROSEDUR RadarLogic():

 recentlySeen ← nearestEnemy ada DAN LastSeen < 5 turn lalu

 JIKA recentlySeen:

 angleToEnemy ← arah dari bot ke posisi musuh
 radarTurn ← normalisasi(angleToEnemy - RadarDirection)
 radarTurn ← radarTurn + (tanda radarTurn × 5) // sedikit overshoot
 SetTurnRadarLeft(radarTurn) // lock ke musuh

LAIN:

SetTurnRadarRight(45) // sweep mencari musuh

PROSEDUR: GunTrackingLogic()

Gun selalu independen dari movement

Arahkan gun ke prediksi posisi musuh,
dengan anti rotate-loop jika gun berputar terlalu lama

PROSEDUR GunTrackingLogic():

JIKA tidak ada nearestEnemy → keluar

recentlySeen ← LastSeen musuh < 8 turn lalu

JIKA recentlySeen:

firePower ← SelectFirePower(musuh, jarak)

(predX, predY) ← PredictEnemyPosition(musuh, firePower)

angleToTarget ← arah ke (predX, predY)

LAIN:

angleToTarget ← arah ke posisi terakhir musuh

gunTurn ← normalisasi(angleToTarget - GunDirection)

// Anti rotate-loop: jika gun terus berputar tanpa hasil

gunDelta ← |normalisasi(GunDirection - lastGunDirection)|

JIKA gunDelta > 1.0:

rotateLoopCounter++

JIKA rotateLoopCounter >= 20:

gunTurn ← arah langsung ke musuh // reset aim

rotateLoopCounter ← 0

LAIN:

rotateLoopCounter ← 0

lastGunDirection ← GunDirection

SetTurnGunLeft(gunTurn)

PROSEDUR: FireLogic()

GREEDY — Tembak segera jika kondisi terpenuhi

Independen dari movement, selalu dicoba tiap turn

PROSEDUR FireLogic():

JIKA tidak ada nearestEnemy → keluar

JIKA GunHeat > 0 → keluar // gun masih panas

distance ← jarak ke nearestEnemy

firePower ← SelectFirePower(nearestEnemy, distance)

gunTurn ← |arah ke musuh - GunDirection|

gunAligned ← gunTurn < 5° // normal: gun hampir lurus

forceFire ← turnsSinceLastFire ≥ 30

DAN gunTurn < 20° // darurat: sudah lama tidak tembak

JIKA gunAligned ATAU forceFire:

SetFire(firePower)

turnsSinceLastFire ← 0

FUNGSI: SelectFirePower(musuh, jarak)

GREEDY — Pilih firepower terbesar yang layak
berdasarkan kondisi musuh dan jarak saat ini

FUNGSI SelectFirePower(musuh, jarak) → double:

JIKA musuh.Energy ≤ 4 → KEMBALIKAN min(3.0, musuh.Energy + 0.1)

JIKA jarak < 150 → KEMBALIKAN 3.0

JIKA jarak < 300 → KEMBALIKAN 2.0

JIKA jarak < 500 → KEMBALIKAN 1.5

KEMBALIKAN 1.0

FUNGSI: PredictEnemyPosition(musuh, firePower)

Estimasi posisi musuh saat peluru tiba
menggunakan iterasi konvergen (linear extrapolation)

FUNGSI PredictEnemyPosition(musuh, firePower) → (x, y):

bulletSpeed ← 20 - (3 × firePower)

travelTime ← jarak ke musuh / bulletSpeed

ULANGI 10 kali: // iterasi konvergensi

px ← musuh.X + VelocityX × travelTime

py ← musuh.Y + VelocityY × travelTime

travelTime ← jarak(bot ke (px,py)) / bulletSpeed

KEMBALIKAN (clamp(px, 20, ArenaW-20),
clamp(py, 20, ArenaH-20))

PROSEDUR: HuntMovement()
Bergerak memutar mencari musuh
Hindari dinding jika terlalu dekat

PROSEDUR HuntMovement():
SmartAvoidWalls()
SetTurnRight(15) // belok kanan pelan
SetForward(150) // maju

PROSEDUR: FightMovement()
Bergerak tegak lurus (perpendicular) terhadap musuh
Ganti arah secara acak setiap 12-28 turn
untuk menghindari peluru musuh

PROSEDUR FightMovement():
JIKA tidak ada nearestEnemy → keluar

moveTurnCounter++
JIKA moveTurnCounter $\geq$ random(12, 28):
 moveDirection $\leftarrow$ moveDirection $\times$ -1 // balik arah
 moveTurnCounter $\leftarrow$ 0
 moveDistance $\leftarrow$ random(70, 160)

bearingToEnemy $\leftarrow$ arah relatif ke musuh
perpendicularAngle $\leftarrow$ bearingToEnemy + 90° // tegak lurus

SetTurnRight(normalisasi(perpendicularAngle))
SetForward(moveDistance $\times$ moveDirection)
SmartAvoidWalls()

PROSEDUR: FleeMovement()
Lari ke pojok terjauh dari musuh
Greedy: pilih pojok dengan jarak MAKSIMUM dari musuh

PROSEDUR FleeMovement():

JIKA tidak ada nearestEnemy → HuntMovement(); keluar

corners ← 4 titik pojok arena (offset 80px dari tepi)

bestCorner ← pojok dengan jarak TERBESAR dari nearestEnemy

SetTurnRight ke arah bestCorner

SetForward(200)

SmartAvoidWalls()

PROSEDUR: DodgeMovement()

Manuver menghindari peluru dengan gerak acak

PROSEDUR DodgeMovement():

moveDirection ← moveDirection × -1

SetTurnRight(random(60°, 120°) × moveDirection)

SetForward(100 × moveDirection)

SmartAvoidWalls()

PROSEDUR: UnstuckRoutine()

3 fase untuk keluar dari kondisi macet:

Fase 0: mundur + rotate

Fase 1: hadap tengah arena

Fase 2: maju ke tengah, lalu trigger combat override

PROSEDUR UnstuckRoutine():

unstuckTurns++

JIKA unstuckPhase == 0: // FASE 0: mundur

SetBack(100)

SetTurnRight(random(45°, 90°) × moveDirection)

JIKA unstuckTurns ≥ 3 → pindah ke fase 1

JIKA unstuckPhase == 1: // FASE 1: hadap tengah

angleToCenter ← BearingToTarget(tengah arena)

SetTurnRight(normalisasi(angleToCenter + random(-45°, 45°)))

JIKA unstuckTurns ≥ 6 → pindah ke fase 2

JIKA unstuckPhase == 2: // FASE 2: maju


```
SetForward(150)
JIKA unstuckTurns  $\geq$  8:
    selesai unstuck
    wallCooldown  $\leftarrow$  10
    JIKA ada musuh  $\rightarrow$  aktifkan combatOverride 15 turn
```

PROSEDUR: SmartAvoidWalls()

Koreksi arah jika terlalu dekat dinding (margin 100px)
Jika kena 2 dinding sekaligus $\rightarrow$ lari ke tengah arena

PROSEDUR SmartAvoidWalls():

JIKA wallCooldown $>$ 0 $\rightarrow$ keluar

```
nearLeft  $\leftarrow$  X  $<$  100
nearRight  $\leftarrow$  X  $>$  ArenaWidth - 100
nearBottom  $\leftarrow$  Y  $<$  100
nearTop  $\leftarrow$  Y  $>$  ArenaHeight - 100
```

JIKA tidak ada yang dekat $\rightarrow$ keluar

```
targetX  $\leftarrow$  clamp(X, 150, ArenaWidth-150)
targetY  $\leftarrow$  clamp(Y, 150, ArenaHeight-150)
```

JIKA kena $\geq$ 2 dinding:

```
targetX  $\leftarrow$  ArenaWidth / 2
targetY  $\leftarrow$  ArenaHeight / 2 // paksa ke tengah
```

```
escapeAngle  $\leftarrow$  BearingToTarget(targetX, targetY)
JIKA |escapeAngle|  $>$  20°  $\rightarrow$  SetTurnRight(normalisasi(escapeAngle))
SetForward(120)
wallCooldown  $\leftarrow$  5
```

PROSEDUR: UpdateStuckDetection()

Deteksi bot diam — hitung selisih posisi tiap turn
Jika $<$ 5px selama 3 turn berturut $\rightarrow$ trigger UNSTUCK

PROSEDUR UpdateStuckDetection():

JIKA prevX $<$ 0 $\rightarrow$ keluar (turn pertama)

```
moved  $\leftarrow$  jarak Euclidean(posisi sekarang, prevX/prevY)
```

JIKA moved < 5 DAN tidak sedang unstucking:

stuckCounter++

JIKA stuckCounter $\geq 3 \rightarrow$ TriggerUnstuck()

LAIN:

stuckCounter $\leftarrow 0$

PROSEDUR: UpdateStateTimeout()

Anti-deadlock: jika > 40 turn di state yang sama

$\rightarrow$ reset paksa, jika HUNT terlalu lama $\rightarrow$ spiral

PROSEDUR UpdateStateTimeout():

JIKA currentState == lastState:

stateHoldTurns++

JIKA stateHoldTurns ≥ 40 :

reset dodging

JIKA state == HUNT $\rightarrow$ SetTurnRight(random) + SetForward(200)

LAIN:

stateHoldTurns $\leftarrow 0$

lastState $\leftarrow$ currentState

EVENT: OnScannedBot

Update data musuh setiap kali radar mendeteksi

Deteksi jika musuh baru menembak $\rightarrow$ trigger dodge

EVENT OnScannedBot(e):

Ambil/buat EnemyInfo untuk e.ScannedBotId

Perbarui: PrevX/Y, X/Y, Direction, Speed, Energy, LastSeen

JIKA musuh.JustFired DAN belum dodging:

dodging $\leftarrow$ true

dodgeTurns $\leftarrow$ random(4, 8)

EVENT OnHitByBullet(e):

dodging $\leftarrow$ true

dodgeTurns $\leftarrow$ random(5, 10) // dodge lebih lama dari deteksi JustFired

EVENT OnHitWall(e):

balik arah, mundur 80, hadap tengah, maju 120
reset semua variabel unstuck, wallCooldown ← 8

EVENT OnHitBot(e):

balik arah, mundur 60 // menabrak musuh → mundur

EVENT OnBotDeath(e):

hapus musuh dari dictionary // bersihkan data musuh yang mati

EVENT OnRoundStarted(e):

reset SEMUA variabel ke kondisi awal

B. Penjelasan Struktur Data, Fungsi, dan Prosedur

Struktur Data:

Bot menggunakan Dictionary<int, EnemyInfo> sebagai penyimpanan utama data musuh, di mana kunci berupa ID bot musuh dan nilainya berupa objek EnemyInfo. Kelas EnemyInfo menyimpan snapshot kondisi musuh hasil scan radar, mencakup posisi sekarang dan sebelumnya (X, Y, PrevX, PrevY), kecepatan (VelocityX/Y yang dihitung dari selisih posisi), energi, dan turn terakhir terlihat (LastSeen). Dari data ini, EnemyInfo juga dapat mendeteksi apakah musuh baru saja menembak melalui properti JustFired, yaitu dengan memeriksa apakah penurunan energi musuh berada dalam rentang biaya peluru (0.09–3.1). Selain itu, bot menggunakan enum BotState untuk merepresentasikan lima kemungkinan perilaku: HUNT, FIGHT, FLEE, DODGE, dan UNSTUCK, yang menjadi dasar pengambilan keputusan greedy setiap turn.

Fungsi

EvaluateState() adalah inti dari algoritma greedy bot — fungsi ini mengevaluasi kondisi saat ini dan mengembalikan state dengan prioritas tertinggi yang kondisinya terpenuhi, mulai dari combat override, unstuck, dodge, flee, fight, hingga hunt sebagai default. GetNearestEnemy() memilih target terbaik dari semua musuh yang aktif menggunakan formula $\text{score} = \text{jarak} + (\text{staleness} \times 50)$, sehingga musuh yang dekat sekaligus baru terdeteksi mendapatkan prioritas tertinggi. SelectFirePower() memilih daya tembak secara greedy berdasarkan jarak dan kondisi energi musuh menggunakan threshold bertingkat. Sementara PredictEnemyPosition() mengestimasi posisi musuh saat peluru tiba dengan melakukan iterasi konvergen sebanyak 10 kali, mengasumsikan musuh bergerak lurus dengan kecepatan konstan.

Prosedur:

1. Main Run

`Run()` adalah main loop yang mengorkestrasikan semua sistem setiap turn. Dalam satu turn, bot memperbarui sistem anti-stuck dan anti-deadlock, mengevaluasi target dan state secara greedy, lalu menjalankan radar, gun tracking, movement, dan fire secara berurutan sebelum memanggil `Go()` untuk mengeksekusi semua perintah sekaligus. Penting untuk dicatat bahwa `RadarLogic()`, `GunTrackingLogic()`, dan `FireLogic()` berjalan independen dari state movement, artinya bot selalu mengarahkan gun dan radar ke musuh serta mencoba menembak di setiap turn, tidak peduli apakah bot sedang HUNT, FLEE, atau bahkan UNSTUCK sekalipun.

2. Movement

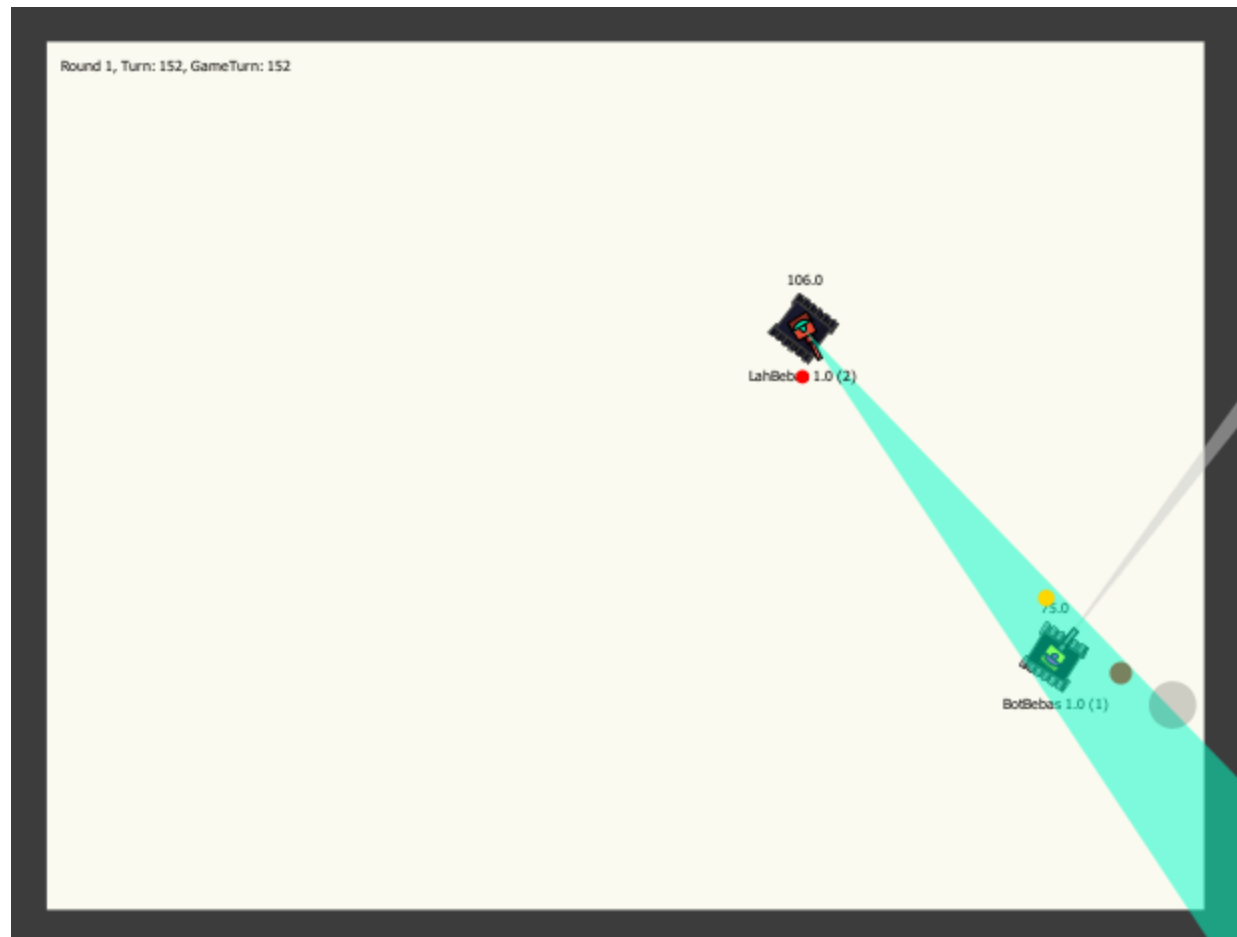
Setiap state memiliki prosedur gerak tersendiri. `FightMovement()` menggerakkan bot secara tegak lurus (strafe) terhadap musuh dengan arah dan jarak yang diganti secara acak setiap 12–28 turn agar pola tidak terprediksi. `FleeMovement()` secara greedy memilih pojok arena yang berjarak paling jauh dari musuh sebagai tujuan pelarian. `DodgeMovement()` melakukan manuver acak untuk menghindari peluru yang sudah terdeteksi, aktif selama 4–10 turn. `HuntMovement()` menggerakkan bot memutar sambil terus maju untuk memperluas coverage pencarian musuh. Semua prosedur movement memanggil `SmartAvoidWalls()` yang mengoreksi arah bot jika terlalu dekat dengan dinding, dengan cooldown 5 turn untuk mencegah koreksi berulang yang menyebabkan gerakan jitter.

3. Anti-Stuck

Bot memiliki tiga lapisan pengaman terhadap kondisi macet. `UpdateStuckDetection()` mengukur jarak perpindahan bot setiap turn dan memicu `UnstuckRoutine()` jika bot bergerak kurang dari 5px selama 3 turn berturut-turut. `UnstuckRoutine()` sendiri bekerja dalam tiga fase terstruktur: mundur sambil rotate, menghadap tengah arena, lalu maju ke tengah. Setelah selesai, bot langsung memaksa state FIGHT melalui mekanisme combat override selama 15 turn. `UpdateStateTimeout()` menangani kasus berbeda, yaitu ketika bot terjebak di satu state terlalu lama (lebih dari 40 turn) meskipun tidak tersangkut secara fisik, dengan mereset paksa kondisi dodging dan memaksa gerakan spiral jika terjadi di state HUNT.

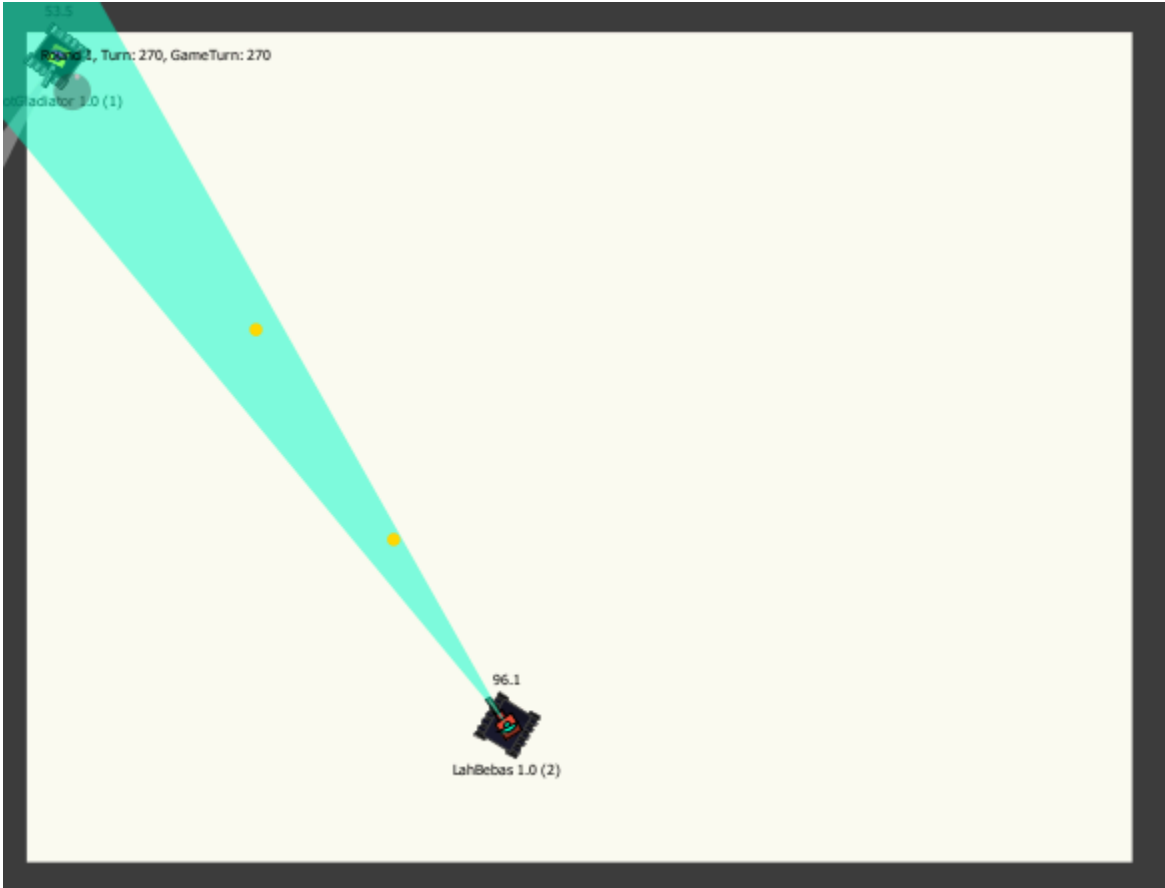
C. Pengujian

1. LahBebas vs BotBebas



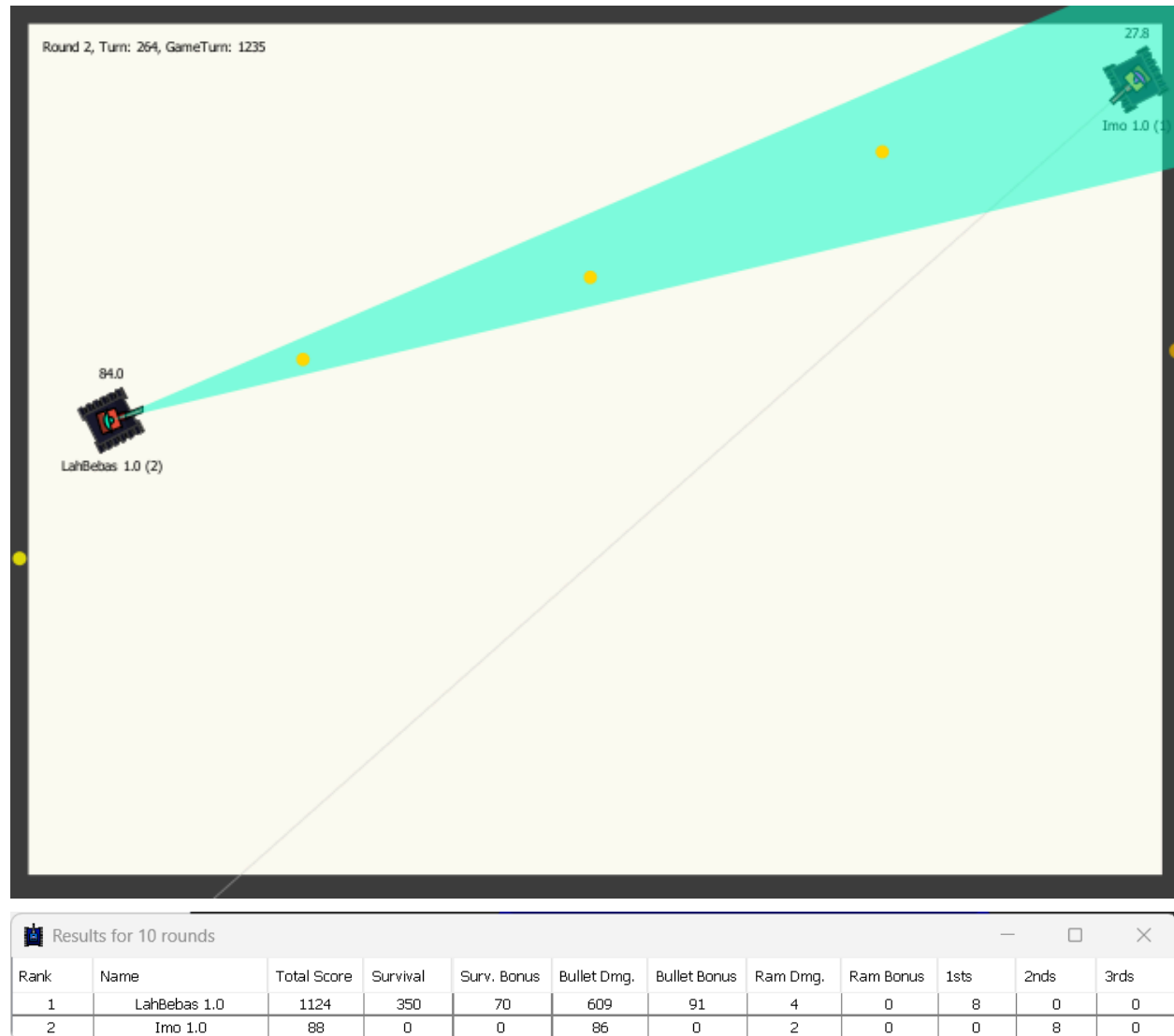
Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	LahBebas 1.0	879	250	50	508	71	0	0	6	0	0
2	BotBebas 1.0	86	0	0	86	0	0	0	0	6	0

2. LahBebas vs BotGladiator



Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	LahBebas 1.0	1152	350	70	594	84	25	28	8	0	0
2	BotGladiator 1.0	174	0	0	148	0	26	0	0	8	0

3. LahBebas vs Imo



D. Analisis Pengujian

Pada simulasi pertarungan 1 vs 1 antar bot LahBebas dengan bot bot lainnya dapat dilihat bahwa bot LahBebas mendominasi pertarungan. Di pertarungan melawan BotBebas, LahBebas berhasil meng lock BotBebas walaupun BotBebas sudah melakukan manuver pergerakan segitiga pada akhirnya kalah juga. Di pertarungan melawan bot BotGladiator pertandingan berakhir lebih cepat, dikarenakan pergerakan BotGladiator lebih agresif dan stuck di bagian sudut pojok tembok sehingga sering menabrak dan membuat energy berkurang. Di pertarungan melawan bot Imo, bot LahBebas dapat dengan mudah meraih kemenangan juga, dikarenakan pergerakan bot yang cukup mudah diprediksi dan gerakan moncong dan scan yang masih kurang tepat.

Bab 5. Kesimpulan dan Saran

Kesimpulan:

Tugas besar ini berhasil mengimplementasikan algoritma Greedy pada empat bot permainan Robocode Tank Royale dengan pendekatan dan heuristik yang berbeda-beda.

Dari keempat bot yang dikembangkan, **LahBebas** terbukti menjadi bot terbaik berdasarkan hasil pengujian 1 vs 1. Bot ini unggul karena mengimplementasikan greedy. Mekanisme FSM dengan 5 state (HUNT, FIGHT, FLEE, DODGE, UNSTUCK) memungkinkan bot beradaptasi secara dinamis terhadap berbagai situasi pertempuran, sesuatu yang tidak dimiliki bot lainnya.

BotBebas dan **Imo** memiliki efisiensi komputasi yang tinggi namun terbatas oleh pola patrol yang mudah diprediksi musuh. **BotGladiator** sebagai varian paling agresif justru menjadi yang paling tidak stabil akibat bug pada parameter gun sweep dan pola zigzag yang tidak memberikan manfaat taktis terukur.

Secara keseluruhan, hasil pengujian membuktikan bahwa strategi greedy yang berlapis dan adaptif, seperti yang diterapkan pada LahBebas lebih efektif dibandingkan strategi greedy tunggal yang statis, terutama dalam menghadapi lawan dengan pola pergerakan yang bervariasi.

Saran:

Saran ditujukan kepada penulis laporan dan orang-orang yang akan mengimplementasikan robocode kedepannya

1. Untuk pengimplementasian kedepannya pada bagian pembuatan strategi sebaiknya explore lebih banyak strategi yang berbeda.
2. Pada bagian implementasi kode pastikan menggunakan function yang sesuai dengan versi robocode yang disediakan.
3. Untuk pembuatan bot harus lebih diperhatikan untuk dictionarynya, karena akan sangat berpengaruh terhadap tingkah laku bot yang sesuai dengan strategi kita.

Lampiran

GitHub:

https://github.com/leec1za/Tubes_LahBebas

Simulasi di Youtube:

https://youtu.be/_gRfQMxUM_Q

No	Poin	Ya	Tidak
1	Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten.	☒	
2	Membuat 4 solusi greedy dengan heuristic yang berbeda.	☒	
3	Membuat laporan sesuai dengan spesifikasi.	☒	
4	Membuat video bonus dan diunggah pada Youtube.	☒	